Terraform Remote State in Azure Storage Cheatsheet

Storing Terraform state files locally causes major issues when collaborating with teams or losing files. Remote state in Azure Storage provides a single source of truth with built-in state locking and encryption.

What you'll learn:

1. Step 1: Create Azure Backend Infrastructure
2. Step 2: Configure Backend in Terraform
3. Step 3: Initialize and Migrate State
4. Step 4: Enable State Locking and Security

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Step 1: Create Azure Backend Infrastructure

Before configuring Terraform, you need an Azure Resource Group, Storage Account, and Blob Container. The storage account holds your state file while Azure handles encryption at rest by default.

```
$az group create --name rg-terraform-state --location eastus

 az storage account create \
   --name sttfstate$RANDOM \
   --resource-group rg-terraform-state \
   --sku Standard_LRS \
   --encryption-services blob

 az storage container create \
   --name tfstate \
   --account-name sttfstate$RANDOM
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Step 2: Configure Backend in Terraform

Add the backend block inside your main.tf or terraform.tf file. This block specifies the exact Azure Storage location where Terraform should read and write state files during execution.

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
  backend "azurerm" {
    resource_group_name  = "rg-terraform-state"
    storage_account_name = "sttfstate12345"
    container_name       = "tfstate"
    key                  = "terraform.tfstate"
  }
}
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Step 3: Initialize and Migrate State

Run terraform init to connect your local workspace to Azure Storage. If you already have a local terraform.tfstate file, Terraform will prompt you to automatically copy it to Azure.

```
$# Authenticate with Azure CLI first
 az login

 # Initialize and push state to Azure Blob Storage
 terraform init
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Step 4: Enable State Locking and Security

Azure Storage natively supports state locking using Blob Leases. When someone runs terraform apply, Azure locks the state file so no one else can modify it simultaneously, preventing state corruption.

```
# Run plan to verify backend works and locks state during execution
terraform plan

# Force unlock ONLY if a deployment crashed and left a stale lock
terraform force-unlock <LOCK-ID>
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always enable soft delete on Azure Blob Storage to recover accidentally deleted state files.
+ Use Azure AD (RBAC) authentication instead of hardcoded storage access keys.
+ Store different environment states (dev, stage, prod) in separate key paths or containers.
+ Never store secret values in clear text; treat the state file as sensitive data.
+ Keep backend infrastructure provisioning separate from application infrastructure code.

Watch Out For

! Committing Azure Storage Account keys or secrets to public GitHub repositories.
! Forgetting to run 'az login' before running 'terraform init' with Azure backend.
! Deleting the backend Storage Account manually while active infrastructure is running.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh